

LangChain 小白指南

零基础拆解 AI Agent 开发框架 —— 不花一分钱、不用 API Key，亲手把一个 AI Agent 拆开看

学习对象: github.com/langchain-ai/langchain | 版本: commit 026c3da (release(openai) 1.6.7)

实测环境: langchain 1.4.3 + langchain_core 1.6.6 + Python 3.14.7 (Windows 11) | 全部实验代码均已实际跑通

疑问即目录 —— 一个工科毕业生第一次接触 LangChain 的 14 个疑问

1. LangChain 到底是什么?
2. 直接调模型 API 不行吗，为什么要它?
3. 这个巨型仓库里都是什么?
4. 术语扫盲: ChatModel / Message / Tool / Agent
5. agent.invoke() 里面到底发生了什么?
6. LLM 又不能执行函数，"工具调用"怎么实现的?
7. @tool 装饰器偷偷做了什么?
8. 模型传错参数还能自动纠正?
9. RAG 是什么? Embeddings / VectorStore 干嘛用?
10. Prompt Template 是什么?
11. LangGraph 是什么? 为什么 v1 拿它当引擎?
12. Middleware 中间件能干什么?
13. LangSmith 和它什么关系?
14. 学完我能用它做什么?

01 LangChain 到底是什么？

仓库 README 自己给的定义是 **"The agent engineering platform"** (Agent 工程平台)。拆成人话：

LangChain 是一个 Python/JS 框架，帮开发者把"大语言模型 + 你的工具 + 你的数据"组装成能自动干活的 AI 应用。

 **讲个故事：**2022 年底 ChatGPT 火了，大家发现"调一下 API、发一段话"就能拿到神奇回答。于是无数人开始写"聊天机器人客服"、"文档问答助手"。但很快每个人都撞上同样一堆破事：想换模型要重写一遍调用代码；想让模型查数据库，得自己发明一套"模型点单 → 我执行 → 结果回填"的协议；多轮对话要自己管理历史记录.....LangChain 的作者 Harrison Chase 在 2022 年 10 月把这些**人人都重复造的轮子**收进一个开源库，就成了 LangChain。到 2025 年 v1 版本，它彻底转向"Agent 工程"——不再只是"调模型的胶水"，而是**造智能体的标准施工平台**。

LangChain 的定位：你写的代码 和 大模型之间的"施工平台"



02 直接调模型 API 不行吗，为什么需要 LangChain?

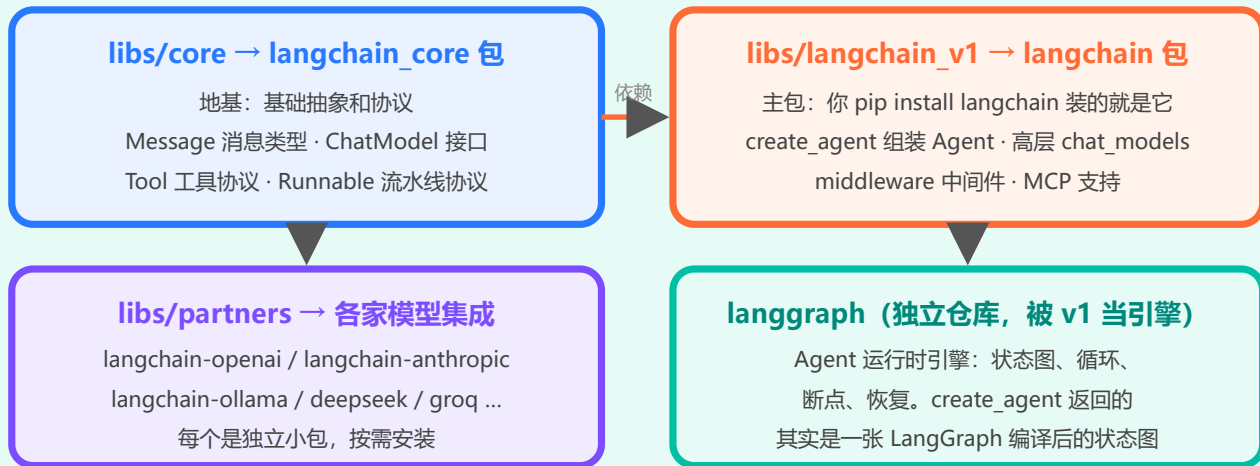
当然"行"——就像不用框架、用 socket 手写 HTTP 也能做网站。问题在于你要自己重复发明一堆轮子，而且每换一家模型就重写一遍。对比一下：

你要做的事	直接调 HTTP API (自己造轮子)	用 LangChain
换个模型供应商	重写请求格式、鉴权、错误处理	改一个字符串： <code>init_chat_model("anthropic:claude-...")</code>
让模型用你的函数 (工具调用)	手工拼 JSON Schema、解析模型返回的"点单"、自己执行再回填	<code>@tool</code> 装饰器 + <code>create_agent</code> 一行组装
Agent 多轮循环 (模型↔工具反复交互)	手写 while 循环 + 停止条件 + 错误兜底	框架内置，且由 LangGraph 引擎保证可靠
流式输出、重试、限流	每个都自己实现	统一内置
RAG (文档问答)	自己选向量库、写切块/嵌入/检索全流程	现成组件拼积木

🎯 **一句话总结：**模型 API 给你的是"一台发动机"，LangChain 给你的是"整车底盘 + 标准化的零件接口"——发动机随便换，车照样开。尤其 v1 之后，它的核心价值聚焦在一件事：把 Agent 的工程可靠性做扎实（循环、状态、纠错、可观测）。

03 这个巨型仓库里都是什么？

LangChain 的 GitHub 仓库是个 monorepo（单仓库多包）。实际克隆后核心就四块：



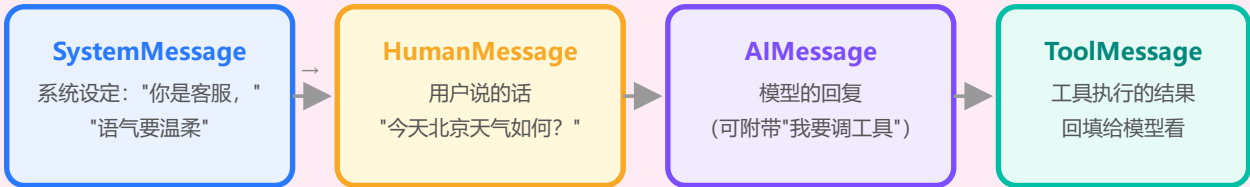
分层记忆口诀： core 定协议（像插座标准），partners 出插头（各家模型），langchain 造电器（Agent），langgraph 是发动机。另外仓库里还有 libs/langchain（0.3 老版本，已冻结维护）和 libs/text-splitters（文本切块工具，RAG 用）。

04 术语扫盲：ChatModel / Message / Tool / Agent

这四个词是整个 LangChain 的"名词地基"，先各给一句话定义：

术语	一句话定义	类比
ChatModel (聊天模型)	把"一段消息列表"变成"一条 AI 回复"的统一接口，屏蔽各厂商差异	万能转接头：不管哪家电器都能插
Message (消息)	对话的基本单元，有 4 种角色 (见下图)	微信聊天的不同"身份气泡"
Tool (工具)	你写的普通 Python 函数，加了说明书 (名字+描述+参数表) 给模型看	菜单上的一道菜：名字和描述写给顾客看
Agent (智能体)	模型 + 工具 + 自动循环：模型自己决定什么时候调哪个工具，循环到任务完成	不只报菜名，还能自己进厨房点菜的顾客

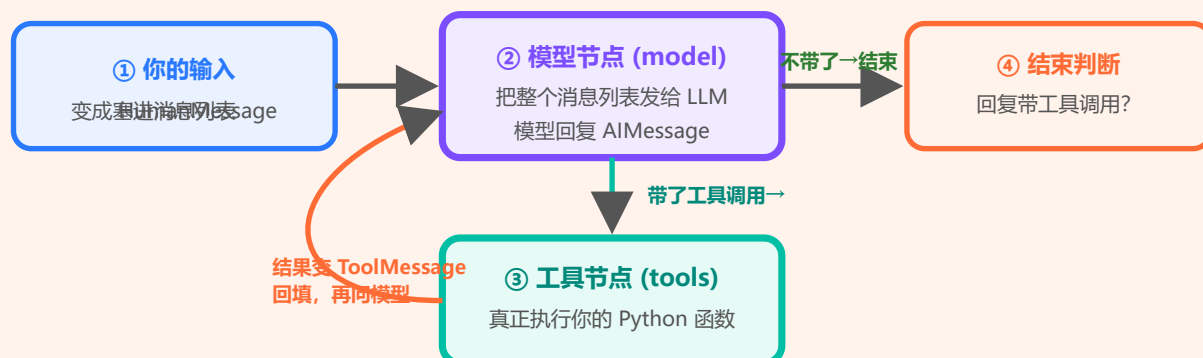
Message 的 4 种角色 (实验 3 实测见到的就是前三种)



05 agent.invoke() 里面到底发生了什么？

这是全篇最核心的一张图。**Agent 就是一个“模型节点”和“工具节点”组成的循环，由 LangGraph 引擎驱动，直到模型不再要求调工具为止：**

agent.invoke() 的内部循环（实验 3 实测验证）



实验 3 跑出来的真实消息流，和上图完全对应：

```
# python exercise/exp3_agent_loop.py 的实测输出
```

```
=== Agent 内部消息流（共 4 条）===
```

- | | |
|---|-----------------------|
| 1. [HumanMessage] 请帮我计算 128+64 | ← ①输入 |
| 2. [AIMessage] '' -> 请求调用工具 add(128,64) | ← ②模型说"我要调工具" |
| 3. [ToolMessage] 192 | ← ③工具真执行了，结果回填 |
| 4. [AIMessage] 128 + 64 = 192。 | ← ④模型看到结果，给出最终回答，循环结束 |

06 LLM 又不能执行函数，“工具调用”到底怎么实现的？

这是初学者最大的误解。**LLM 全程只做一件事：读文字、吐文字。它从来不能执行你的代码。**“工具调用”的真相是一场三方接力：

工具调用 = LLM“点菜” + 框架“炒菜” + 结果“上菜”

🗣️ LLM (只会说话)

看到工具说明书后输出：

```
{"name": "add",  
"args": {"a": 128, "b": 64}}
```

——只是一段结构化文字！

⚙️ 框架 (LangGraph 工具节点)

解析这段 JSON，
在

你的电脑上真正调用 `add(128, 64) → 192` 🔄 回填循环 把 192 包成 `ToolMessage` 追加进消息列表，再发给 LLM 看 虚线：结果最终回到 LLM 的“眼睛”里

`bind_tools` 做的事：把工具说明书 (JSON Schema) 随请求发给 LLM，让 LLM 知道“有哪些菜可以点”。安全上注意：能被 Agent 调的函数就等于交给了模型“决策、你来执行”，所以危险操作（删文件、转账）一定要在工具函数内部加确认或限制。

07 @tool 装饰器偷偷做了什么？

@tool 把一个普通函数加工成“带说明书的工具”：函数名 → 工具名，docstring → 功能描述，类型注解 → 参数表 (JSON Schema)。这三样合起来就是发给 LLM 的“菜单”。实验 2 的实测输出：

```
# python exercise/exp2_make_tool.py 的实测输出
工具名: get_weather
给 LLM 看的说明书: 查询指定城市的当前天气。

Args:
  city: 城市名，例如"北京"
参数说明书: {'description': '查询指定城市的当前天气。...', 'properties':
  {'city': {'title': 'City', 'type': 'string'}}, 'required': ['city'],
  'title': 'get_weather', 'type': 'object'}
直接调用结果: 北京: 晴, 26°C
```

💡 **为什么 docstring 和类型注解如此重要：**它们不是写给人看的注释，而是写给 LLM 看的唯一说明书。LLM 拿不到你的函数体，它决定调不调、怎么调，全凭这份说明书。所以“docstring 含糊 → 模型乱调工具”是真实会发生的故事。**写工具 = 一半写代码，一半写文案。**

08 模型传错参数还能自动纠正？——Agent 的自我纠错循环

实验 4 里我们让“模型”第一次故意把除数传成字符串 `"zero"`，结果 Agent 没崩，而是自己改对了。原理：**工具执行/参数校验失败时，框架不抛异常中断，而是把错误文字包成 ToolMessage 回流给模型——模型读到错误说明，下一轮就自己改。**实测消息流：

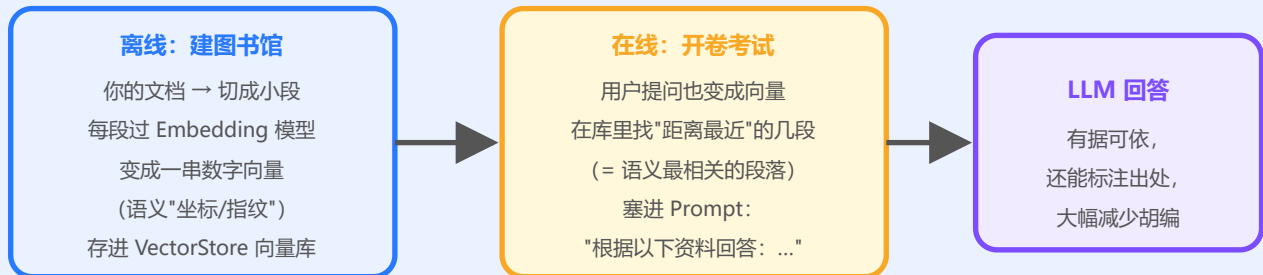
```
# python exercise/exp4_self_heal.py 的实测输出（6 条消息）
[HumanMessage] '请计算 100 除以 4'
[AIMessage] ''                                     ← 模型第一次手滑：b 传了 "zero"
[ToolMessage] "Error invoking tool 'divide' with kwargs {'a': 100,
    'b': 'zero'} with error: b: Input should be a valid number..."
    ← 错误说明回流给模型（关键！）
[AIMessage] ''                                     ← 模型读完错误，改传 b=4 重试
[ToolMessage] '25.0'
[AIMessage] '100 ÷ 4 = 25'                          ← 成功
```

 **和知识库的关联：**这正是我们已掌握的 **[[质检Gate与自我纠错循环]]** 的 LLM 版——“生成（模型决定调用）→ 检查（参数校验/执行）→ 报错回传 → 修正再生成”。区别在于：质检 Gate 里的裁判是**确定性脚本**（参数校验是 100% 确定的），被纠错的却是“不确定的”LLM——**用确定性的 Gate 去驯服不确定的生成者**，和我们学过的架构思想完全同构。另外注意：默认兜底只覆盖**参数校验错误**（包装成 `ToolInvocationError`）；你在工具函数里主动 `raise` 的自定义异常默认会直接炸掉循环，需要在工具内自己 `try/catch` 成返回错误文字。

09 RAG 是什么? Embeddings / VectorStore 干嘛用?

RAG (Retrieval-Augmented Generation, 检索增强生成) 解决一个根本问题: **模型没读过你公司的内部文档, 而且上下文窗口塞不下整个文档库**。RAG 的思路是"开卷考试": 先检索出最相关的几段, 塞进提示词再让模型回答。

RAG = 建图书馆 (离线) + 开卷查资料 (在线)



Embedding (嵌入) 一句话: **把一段文字变成一个高维坐标点, 意思越近的文字坐标越近**——"小猫"和"猫咪"的坐标几乎重合, 和"股票"相距很远。**VectorStore (向量库)** 就是专门存这些坐标点、并支持"找最近邻"的数据库 (Chroma、Qdrant 等, 仓库 partners/ 里都有集成)。core 里的 `embeddings/` 和 `vectorstores/` 目录就是这两者的统一抽象。

10 Prompt Template (提示词模板) 是什么?

就是 **带变量槽位的提示词字符串**——把"固定话术"和"每次变化的内容"分离:

```
# 没有模板: 字符串拼接到处飞, 改一句话术要翻遍代码
prompt = "你是一名" + role + "客服。请根据以下资料回答问题: \n" + docs + "\n问题: " + q

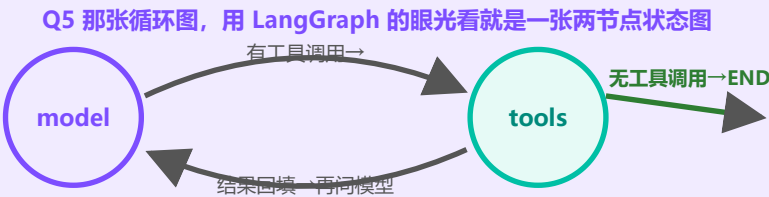
# 有模板: 话术集中管理, 变量显式声明
from langchain_core.prompts import ChatPromptTemplate
tpl = ChatPromptTemplate.from_messages([
    ("system", "你是一名{role}客服, 请根据资料回答, 资料: {docs}"),
    ("human", "{question}"),
])
msgs = tpl.invoke({"role": "售后", "docs": "...", "question": "怎么退货?"})
```

它还有两个隐藏价值: ① **变量缺失/拼错在运行前就报错**; ② **模板本身是可复用、可测试的对象** (还记得知识库里的"确定性脚本"吗——模板让提示词的组装过程变成确定的)。

11 LangGraph 是什么？为什么 v1 拿它当引擎？

LangChain v1 的 `create_agent` 返回值类型是 `CompiledStateGraph` ——你拿到的 Agent 本质上是一张 LangGraph 编译后的“状态图” (state machine)。

🔗 为什么要图：简单的 while 循环只能“模型↔工具转到天荒地老”。真实 Agent 需要更复杂的控制流：中途人工审批 (human-in-the-loop)、失败回退到上一个节点、长任务断点续跑 (checkpointer 持久化状态)、多个 Agent 协作。**图 = 节点 (干活的步骤) + 边 (跳转规则) + 全局状态 (所有节点共读共写的消息列表等)**，这些高级能力全都有了着落。这和我们知识库的 [[构建流水线与Pass]] 是同一个思想：复杂工作拆成顺序阶段——只不过 LangGraph 的“流水线”允许分叉、回路和暂停。



12 Middleware (中间件) 能干什么？

v1 的 `create_agent(middleware=[...])` 参数允许你在 Agent 循环的各个环节“插一脚”：**模型调用前后、工具调用前后、循环结束时的钩子**。类比知识库的 [[构建流水线与Pass]]——中间件就是往 Agent 主循环这条流水线里插入自定义 Pass。

官方内置的中间件 ([langchain/agents/middleware/](#) 目录) 展示了典型用途：

内置中间件	干什么
summarization	对话太长时自动压缩成摘要，省上下文窗口 (长期记忆)
human-in-the-loop	危险工具调用前暂停，等人类点“批准”再继续
PII / 敏感信息处理	发给模型前先抹掉手机号、身份证号等隐私
tool call 修正	拦截并修改模型的工具调用参数 (如强制覆盖某个参数)

13 LangSmith 和它什么关系？

LangSmith 是 LangChain 公司的商业产品（不在本仓库里），定位是 Agent 的“行车记录仪 + 考官”：

- ① **可观测**：Agent 循环十几轮、嵌套调工具，出问题时你亟需看到“每一轮模型到底收到什么、吐了什么、哪个工具慢了”——LangSmith 把每次 invoke 的完整调用树记录下来供回放调试。
- ② **评估**：改了提示词效果变好还是变坏？跑一个数据集打分对比。
- ③ **部署**：把 Agent 托管成服务。

观点：Agent 是“不确定的系统”，传统断点调试很无力——你不知道模型这轮为什么突然抽风。**可观测性（把每一次决策留痕）对 Agent 的重要性，远超传统软件**。开源部分 LangChain 也自带轻量的回调/追踪机制（core 的 `tracers/`），LangSmith 是它的商业增强版。

14 学完我能用它做什么？

LangChain 提供的能力清单 + 结合当前水平的具体用途：

能力	你能做什么（由易到难）
统一模型接口	① 命令行聊天机器人 ：30 行代码，随时在 openai/anthropic/ollama(本地免费) 之间切换
@tool + create_agent	② 个人工具助手 ：让它查本地文件、算数、调你写的任何 Python 函数
RAG 组件	③ 文档问答 ：把 PDF 笔记/公司文档喂进去，“开卷考试”式问答
结构化输出	④ 信息抽取器 ：从一堆邮件/网页里抽 JSON 字段（发票号、日期、金额）
middleware + checkpointer	⑤ 带审批的自动化 ：危险操作前暂停等人工确认，长任务断点续跑
langgraph 图编排	⑥ 多 Agent 协作系统 ：规划 Agent + 执行 Agent + 审查 Agent 各司其职

建议路线：先做 ①②（本指南实验已带你走完核心原理），再挑战 ③④，⑤⑥ 等有真实需求再上。所有用途都能先用假模型离线开发调试，最后一步才接真模型花钱。

 **主实验：零成本拆解一个 AI Agent**

5 个步骤全部不联网、不用 API Key、不花一分钱；所有命令与输出均已在本机 (F:\reminder\langchain\exercise\) 实际执行验证

步骤 1 和“假 LLM”说第一句话 —— 认识 ChatModel 与 Message

🔗 目的：体验“统一模型接口”；🔗 实现：用 FakeListChatModel 冒充 LLM（返回预设回答），跑通 model.invoke() 最小闭环。

```
# 终端执行（在 F:\reminder\langchain 目录下）：  
.\.venv\Scripts\python.exe exercise\exp1_hello_model.py
```

```
# exercise\exp1_hello_model.py 的内容：  
from langchain_core.language_models.fake_chat_models import FakeListChatModel  
# 假模型：按顺序返回预设回答。换成 init_chat_model("openai:gpt-5.5") 就是真模型  
model = FakeListChatModel(responses=["你好！我是你的第一个大模型。"])  
result = model.invoke("你好，请介绍一下你自己")  
print("回答:", result.content)
```

✅ 预期结果（实测）：

回答：你好！我是你的第一个大模型。

消息类型：AIMessage —— invoke 返回的不是字符串，是 AIMessage 对象。

步骤 2 用 @tool 造一把“工具” —— 看函数如何变成给模型看的菜单

🔗 目的：亲眼看到 @tool 生成的说明书；🔗 实现：装饰一个查天气函数，打印它的 name / description / 参数 Schema。

```
.\.venv\Scripts\python.exe exercise\exp2_make_tool.py
```

✅ 预期结果（实测）：打印出工具名 `get_weather`、docstring 变成的描述文字、以及 JSON Schema ({'properties': {'city': {'type': 'string'}}, 'required': ['city'], ...}) ——这份 Schema 就是 LLM 决定“怎么传参”的唯一依据。

步骤 3 create_agent 组装完整 Agent —— 亲眼看见消息循环

🎯 目的：观察 Q5 讲的内部循环；🔧 实现：假模型按"剧本"先调工具再作答，打印 Agent 内部全部消息。

```
.\.venv\Scripts\python.exe exercise\exp3_agent_loop.py
```

✅ 预期结果（实测）：4 条消息——HumanMessage（提问）→ AIMessage（请求调 add(128,64)）→ ToolMessage（192）→ AIMessage（最终回答）。中间那两步"点单→执行→回填"全是框架自动完成的。

步骤 4 搞破坏：让模型传错参数 —— 亲眼看 Agent 自我纠错

🎯 目的：验证"错误回流"机制；🔧 实现：剧本里模型第一次把除数传成字符串 "zero"，观察参数校验错误如何被包成 ToolMessage 送回模型。

```
.\.venv\Scripts\python.exe exercise\exp4_self_heal.py
```

✅ 预期结果（实测）：6 条消息，第 3 条是错误说明（`Error invoking tool 'divide' ... Input should be a valid number...`），随后模型自动改传 `b=4` 重试成功。⚠️ 注意：若你在工具函数里 raise 自定义异常（如 `ValueError`），默认会直接中断——校验错误才被自动兜底。

步骤 5 流式输出 —— 看见"打字机效果"的底层机制

🎯 目的：理解 ChatGPT 式逐字输出的原理；🔧 实现：`model.stream()` 返回的是 `AIMessageChunk` 片段序列，逐个打印。

```
.\.venv\Scripts\python.exe exercise\exp5_stream.py
```

✅ 预期结果（实测）：单词被空格切分逐段输出：`Hello world this is streaming`——每个片段是一个 `AIMessageChunk`，前端就是边收边渲染。

🚀 进阶卡 (可选, 需自备 API Key —— 本步未实际执行, 语法来自官方 README)

想接真模型, 只需两步: ① `pip install -U langchain-openai` (或 `anthropic` / `ollama` 等对应集成包); ② 环境变量放好 key (如 `OPENAI_API_KEY`), 然后把假模型换成一行:

```
from langchain.chat_models import init_chat_model
model = init_chat_model("openai:gpt-5.5") # 或 "anthropic:claude-..." / "ollama:qwen3"
```

实验 3/4 的 Agent 代码其余部分**一行都不用改**——这就是统一接口的意义。先用 `ollama` 本地模型可以零成本试真模型。